Aim:

To implement QR decomposition algorithm using the Gram-Schmidt method.

Equipment's required:

1. Hardware – PCs
2. Anaconda – Python 3.7 Installation / Moodle-Code Runner

Algorithm:

1. Intialize the matrix Q and u
2. The vector u and e is given by

$$u_1 = a_1$$

$$u_2 = a_2 - (a_2.e_1)e_1$$

$$u_3 = a_3 - (a_3.e_1)e_1 - (a_3.e_2)e_2$$

$$e_1 = \frac{u_1}{\|u_1\|}$$

$$e_2 = \frac{u_2}{\|u_2\|}$$

3. Obtain the Q matrix

$$Q = (e_1|e_2| \dots \dots \dots e_n)$$

4. Construct the upper triangular matrix R

$$R = \begin{bmatrix} a_1.e_1 & a_2.e_1 & a_3.e_1 \\ 0 & a_2.e_2 & a_3.e_2 \\ 0 & 0 & a_3.e_3 \end{bmatrix}$$

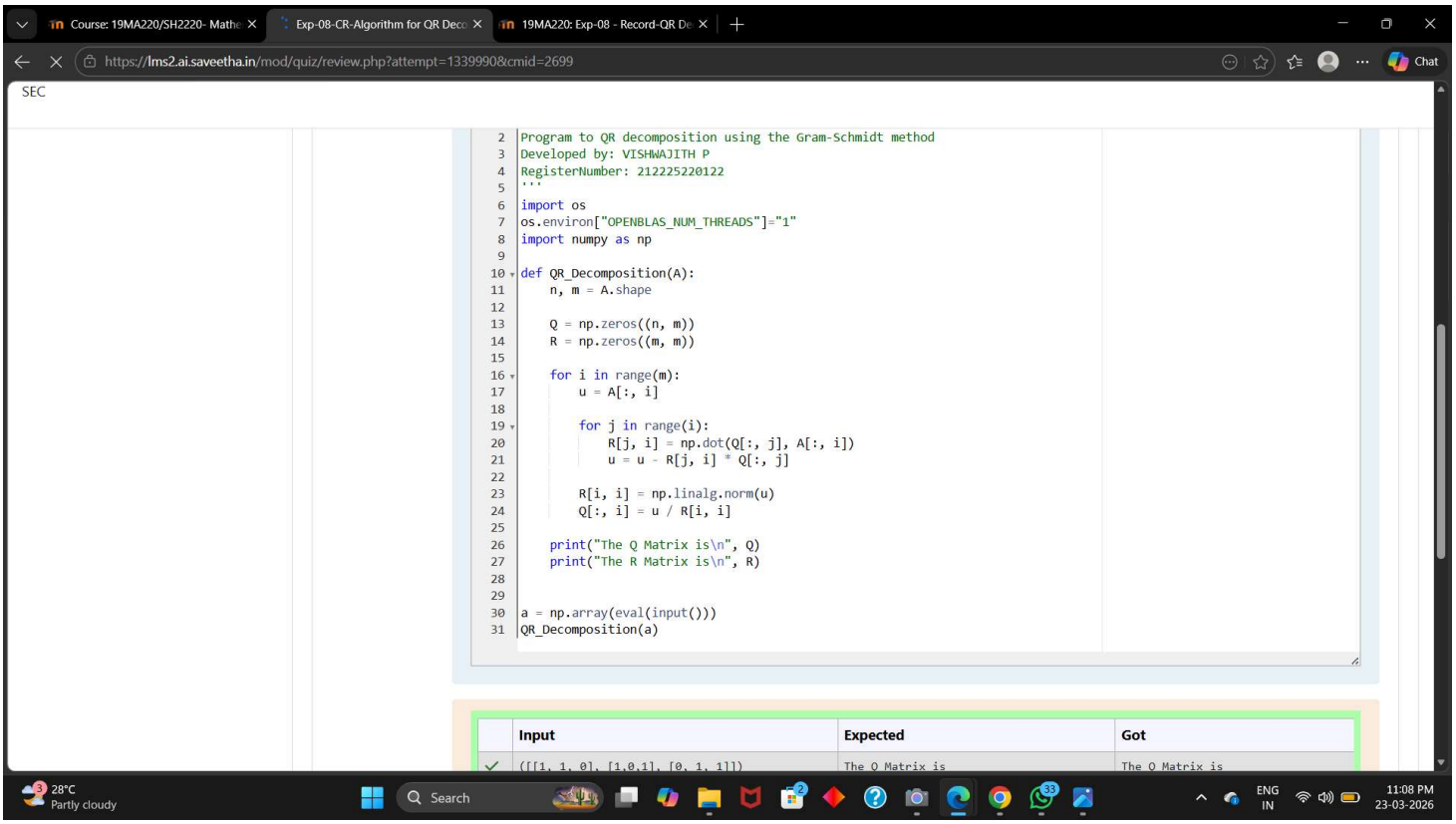
Program:

Gram-Schmidt Method

```
import os
os.environ["OPENBLAS_NUM_THREADS"]="1"
```

```
import numpy as np

def QR_Decomposition(A):
    n, m = A.shape

    Q = np.zeros((n, m))
    R = np.zeros((m, m))

    for i in range(m):
        u = A[:, i]

        for j in range(i):
            R[j, i] = np.dot(Q[:, j], A[:, i])
            u = u - R[j, i] * Q[:, j]

        R[i, i] = np.linalg.norm(u)
        Q[:, i] = u / R[i, i]

    print("The Q Matrix is\n", Q)
    print("The R Matrix is\n", R)


a = np.array(eval(input()))
QR_Decomposition(a)
```

Output



Result

Thus the QR decomposition algorithm using the Gram-Schmidt process is written and verified the result.